

Architecture de cyberMans — Lab Platform

Document d'accueil et de référence. Il vise deux choses : qu'une personne qui rejoint le projet sache où poser les yeux au bout d'une heure, et qu'elle sache où poser son code au bout d'une journée.

Version du document : 26 septembre 2026 · Code décrit : branche `master`

1. Ce que fait la plateforme

cyberMans est une plateforme d'entraînement à la cybersécurité, dans l'esprit de Hack The Box, avec une partie cours dans l'esprit d'OpenClassrooms.

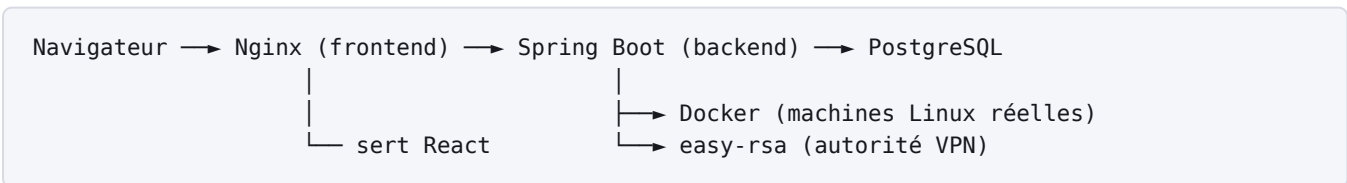
Un utilisateur inscrit dispose de :

- **deux machines d'attaque personnelles** — une Windows (RDP), une Linux (SSH) — qu'il démarre et arrête, avec des identifiants temporaires régénérés à chaque démarrage ;
- **un profil VPN personnel** (`.ovpn`) pour joindre le réseau du lab ;
- **un catalogue de machines à compromettre**, deux flags par machine, des points, un rang, un classement, des hauts faits ;
- **des cours** répartis en deux filières, forensique et défense, avec suivi de lecture ;
- et, s'il est administrateur, **un tableau de bord** pour publier ces cours.

Les trois conteneurs

Service	Image	Rôle
db	<code>postgres:17-alpine</code>	Base de données. Schéma appliqué au démarrage.
backend	build de <code>backend/Dockerfile</code>	API REST. Non exposée sur l'hôte.
frontend	build de <code>frontend/Dockerfile</code>	Nginx : sert l'application et relaie <code>/api</code> vers le backend.

Le navigateur ne parle qu'à Nginx. Même origine pour l'application et l'API, ce qui simplifie la sécurité (voir §10).



2. Le principe : architecture hexagonale

Le backend suit l'architecture hexagonale, aussi appelée *ports et adaptateurs*. En une phrase :

Le cœur métier ne connaît personne ; ce sont les autres qui le connaissent.

Concrètement, trois règles tiennent tout le reste.

1. Le domaine ne dépend d'aucun framework. Aucune classe de `domain/` n'importe Spring, JPA, Jackson ou HTTP. On peut lire une règle métier sans connaître la technique, et l'exécuter dans un test sans démarrer de serveur.

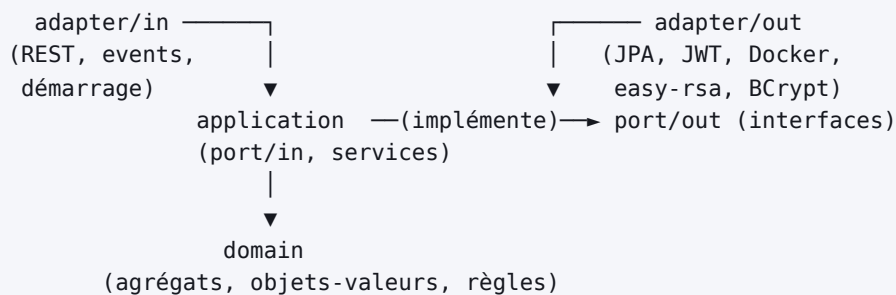
2. Les besoins techniques sont exprimés par des interfaces que l'application possède. L'application dit « j'ai besoin de sauvegarder un utilisateur » (`UserRepositoryPort`) sans dire comment. C'est l'inversion de dépendance : la flèche de dépendance pointe vers le métier, pas vers la base.

3. La technique vit dans des adaptateurs, à la périphérie. Un adaptateur *entrant* provoque un cas d'usage (un contrôleur REST, un écouteur d'événement, un démarrage). Un adaptateur *sortant* réalise un besoin (JPA, BCrypt, Docker, easy-rsa).

Ce que cela achète

Bénéfice	En pratique dans ce dépôt
Tests rapides	107 des 126 tests tournent sans Spring ni base, en quelques millisecondes.
Changer d'infrastructure	Deux hyperviseurs coexistent (<code>simulated</code> , <code>docker</code>) derrière un même port ; passer de l'un à l'autre est une variable d'environnement.
Règles lisibles	<code>VirtualMachine.markStarted()</code> dit tout de la machine à états, en vingt lignes, sans SQL.
Frontières nettes	Une régression de couche se voit à la compilation : le domaine ne compile pas s'il importe Spring.

Le sens des dépendances



Une flèche signifie « connaît ». Aucune ne part du domaine.

3. Les quatre couches du backend

Le code vit sous `backend/src/main/java/com/labplatform/`.

Couche	Fichiers	Contenu	Peut dépendre de
domain/	46	Agrégats, objets-valeurs, politiques, exceptions métier	rien (JDK seulement)
application/	71	Cas d'usage (port/in), besoins (port/out), services	domain
adapter/	72	REST, JPA, sécurité, hyperviseur, VPN, démarrage	application , domain
config/	3	Racine de composition, sécurité HTTP, propriétés	tout

Règle de revue : un import de `org.springframework` ou de `jakarta.persistence` dans `domain/` est un défaut, pas un détail. C'est le seul invariant d'architecture à vérifier systématiquement en relecture.

Lecture rapide de l'arborescence

```

com.labplatform
├── domain/
│   ├── user/
│   │   ├── Le métier, en Java pur
│   │   │   ├── User, Email, Role, PasswordPolicy, PasswordResetToken,
│   │   │   │   Actor, AdminPolicy, UserRegistered
│   │   └── lab/
│   │       ├── VirtualMachine (machine à états), ConnectionInfo,
│   │       │   OperatingSystem, AccessProtocol, VmStatus,
│   │       │   VmAccessPolicy, LabTemplate
│   │       └── box/
│   │           ├── Box (agrégat), Flag, Difficulty, FlagKind, Own,
│   │           │   BoxRating, CommunityRating
│   │           ├── scoring/
│   │           │   Rank, PlayerProgress, PlayerScore, Handle
│   │           └── academy/
│   │               ├── Course (agrégat), CourseSection, CourseProgress,
│   │               │   SectionCompletion, Track, CourseLevel, SectionKind, Slug
│   │               └── achievement/
│   │                   Achievement (règles), PlayerRecord
│   └── vpn/
│       ├── VpnProfile, VpnEndpoint, VpnProtocol
│       └── shared/
│           DomainException et ses catégories
├── application/
│   ├── port/in/
│   │   Ce que la plateforme sait faire (par contexte)
│   ├── port/out/
│   │   Ce dont elle a besoin (16 interfaces)
│   └── service/
│       Les implémentations des cas d'usage (12 classes)
├── adapter/
│   ├── in/web/
│   │   Contrôleurs REST, DTO, sécurité HTTP, erreurs
│   ├── in/event/
│   │   Provisionnement du lab à l'inscription
│   ├── in/startup/
│   │   Semis du catalogue, des cours, promotion des admins
│   └── out/
│       persistance, security, hypervisor, vpn,
│       transaction, event, notification, process
└── config/
    UseCaseConfig, SecurityConfig, AppProperties

```

4. Le domaine, contexte par contexte

Chaque contexte porte un invariant : la chose qui doit rester vraie, quoi qu'il arrive. C'est le meilleur angle pour comprendre le code.

user — qui est là, et avec quels droits

- **User** — agrégat. Le mot de passe n'y figure que sous forme d'empreinte ; le hachage est délégué à un port. `resetPassword()` refuse un jeton absent, faux ou expiré, et un changement de mot de passe invalide tout jeton en cours.

- **Email** — objet-valeur normalisé (trim, minuscules) et validé à la construction. `Alice@x.io` et `alice@x.io` désignent le même compte, partout, sans que personne ait à y penser.
- **Actor** — *qui agit* : (`userId`, `role`), construit par l'adaptateur d'entrée à partir du jeton. Tous les cas d'usage le prennent en premier paramètre. C'est l'unique source d'identité côté métier.
- **AdminPolicy.requireAdmin(actor)** — la règle « seul un administrateur publie du contenu », exprimée une fois.

lab — les machines d'attaque personnelles

- **VirtualMachine** — machine à états `STOPPED` $\rightleftharpoons$ `RUNNING` portant un invariant fort : *des informations de connexion existent si et seulement si la machine tourne*. Le constructeur refuse tout état incohérent, et la même contrainte est doublée en base (`ck_vm_state`).
- **VmAccessPolicy** — un utilisateur n'opère que ses machines ; un refus est présenté comme une absence (**404**, pas **403**) pour ne pas révéler l'existence des machines d'autrui.

box — les machines à compromettre

- **Box** — agrégat, **seul détenteur des deux flags**. C'est lui qui juge une soumission (`claim()`) et frappe la possession. Aucun service ne compare de flag : la règle n'a qu'un seul endroit où être fausse.
- **Flag** — objet-valeur ne conservant qu'une empreinte SHA-256. Comparaison en temps constant (`MessageDigest.isEqual`), saisie tolérante (espaces, casse, habillage `CYBM{...}`), et `toString()` qui ne laisse rien fuiter dans les journaux.
- **Difficulty** — le barème, et lui seul : 40 % pour le flag utilisateur, 60 % pour le root. Aucun nombre de points n'est écrit ailleurs.
- **Own** — possession d'un flag, **points figés** à la validation : rééquilibrer une machine ne réécrit pas le passé des joueurs.
- **BoxRating** / **CommunityRating** — la difficulté ressentie. `cast()` refuse un vote sur une machine non possédée.

scoring — se situer

- **Rank** — le rang se mesure en **part du catalogue possédée**, pas en total absolu : publier de nouvelles machines ne dégrade le rang de personne, et le sommet reste atteignable quelle que soit la taille du catalogue.
- **Handle** — le classement est public, donc il n'y figure qu'un pseudonyme dérivé de la partie locale de l'e-mail, jamais l'adresse.

academy — les cours

- **Course** — agrégat qui **possède ses sections** : il les ordonne, répond de leur existence (`requireSection()`) et calcule l'avancement d'un apprenant à partir des seules sections cochées.
- **CourseSection** — texte, vidéo, ou les deux. L'adresse d'une vidéo est normalisée et doit être en `http(s) : ni javascript: , ni data: .`
- **Slug** — fabrique d'identifiants d'URL à partir d'un titre saisi (« Réponse à incident » → `reponse-a-incident`), avec variantes uniques.
- **Track** — une filière = une constante. En ajouter une ne touche ni les cas d'usage, ni l'API.

achievement — les hauts faits

- **Achievement** — une énumération où chaque constante porte sa condition (`Predicate<PlayerRecord>`). **Rien n'est stocké** : les hauts faits se déduisent du palmarès à chaque lecture, donc ils ne peuvent ni manquer, ni se perdre, ni se décerner deux fois — et en ajouter un n'exige aucune migration.

shared — le vocabulaire des erreurs

Cinq catégories, traduites une seule fois en codes HTTP (§6) :

Exception	Statut	Sens
<code>InvalidInputException</code>	400	La donnée viole une règle métier
<code>AuthenticationFailedException</code>	401	Identité non prouvée
<code>ForbiddenException</code>	403	Identité prouvée, droits insuffisants
<code>NotFoundException</code>	404	Inexistant — ou invisible pour cet appelant
<code>ConflictException</code>	409	Incompatible avec l'état actuel
<code>ServiceUnavailableException</code>	503	Dépendance externe en panne

5. La couche application

port/in — ce que la plateforme sait faire

Une interface par cas d'usage, groupée par contexte. Elles constituent le catalogue exhaustif des actions possibles : les lire, c'est lire le produit.

Contexte	Cas d'usage
auth	RegisterUser, AuthenticateUser, RequestPasswordReset, ResetPassword
account	GetUserProfile, ChangePassword
lab	ListVms, GetVmInfo, StartVm, StopVm, GetVmConsole, ProvisionDefaultLab
box	ListBoxes, GetBox, SubmitFlag, RateBox
scoring	GetPlayerProgress, GetLeaderboard
academy	ListCourses, GetCourse, TrackSectionProgress, GetLearningProgress
profile	GetAchievements, GetActivity
admin	ManageCourses, GetAdminOverview
vpn	GetVpnAccess, DownloadVpnProfile, RegenerateVpnProfile, GetVpnRevocationList

port/out — ce dont elle a besoin

Seize interfaces, que les adaptateurs réalisent.

Port	Ce qu'il abstrait	Réalisation
UserRepositoryPort	Comptes	JPA / PostgreSQL
VirtualMachineRepositoryPort	Machines personnelles	JPA
BoxRepositoryPort , OwnRepositoryPort , BoxRatingRepositoryPort	Catalogue, possessions, votes	JPA
CourseRepositoryPort , SectionCompletionRepositoryPort	Cours, suivi de lecture	JPA
VpnProfileRepositoryPort	Certificats clients	JPA
BoxInstanceRepositoryPort	Cibles lancées à la demande	JPA
MediaAssetRepositoryPort / MediaStoragePort	Fiches et contenus des fichiers téléversés	JPA / système de fichiers
QuizRepositoryPort	Questions et propositions des quiz	JPA
WriteupRepositoryPort	Comptes rendus de compromission	JPA
PasswordHasherPort	Hachage	BCrypt (coût 12)
AccessTokenIssuerPort	Émission de jeton de session	JWT (HMAC)
SecretGeneratorPort	Aléa sûr (jetons, flags)	SecureRandom
HypervisorPort	Allumer/éteindre une machine	simulé ou Docker
VpnCertificateAuthorityPort	Émettre/révoquer un certificat	easy-rsa
PasswordResetNotifierPort	Envoyer un lien	journalisation (à brancher)
DomainEventPublisherPort	Publier un événement métier	ApplicationEventPublisher
TransactionPort	Frontière transactionnelle	TransactionTemplate

TransactionPort mérite une note : **l'application décide de ce qui doit être atomique, l'adaptateur décide comment**. C'est ce qui permet d'écrire `transactions.inTransaction(() -> ...)` dans un service sans annotation Spring, et de tester ce service avec une transaction factice.

service — l'orchestration

Douze classes, **sans aucune annotation Spring** : elles sont instanciées à la main dans `UseCaseConfig` (§7). Un service fait trois choses — charger, demander au domaine de décider, persister — et rien d'autre.

Service	Cas d'usage servis
AuthenticationService	Inscription, connexion
PasswordResetService	Demande et réinitialisation
AccountService	Profil, changement de mot de passe
LabService	Provisionnement, démarrage, arrêt, console
BoxService	Catalogue, soumission de flag, notation
ScoreboardService	Progression, classement
AcademyService	Cours, suivi de lecture
CourseAdminService	Publication des cours, vue d'ensemble
ProfileService	Hauts faits, activité
VpnService	Profils .ovpn , révocation

Un exemple de discipline : dans `LabService` , l'appel à l'hyperviseur est fait **hors transaction**. Démarrer un conteneur prend plusieurs secondes ; une connexion SQL ne doit pas rester ouverte pendant ce temps. Le service relit donc l'état, agit, puis rouvre une transaction courte pour enregistrer.

6. Les adaptateurs

Entrants

adapter/in/web — neuf contrôleurs REST, minces par construction : ils traduisent HTTP → cas d'usage → DTO, sans logique.

Contrôleur	Préfixe
AuthController	/api/auth
AccountController	/api/users/me
LabController	/api/labs/vms
BoxController	/api/boxes
ScoreboardController	/api/scoreboard
CourseController	/api/courses
ProfileController	/api/profile
AdminCourseController	/api/admin
VpnController	/api/vpn

Le **mappage DTO est explicite, jamais automatique**. Ce n'est pas du zèle : `Box` porte ses deux flags, et sérialiser l'agrégat les publierait. Chaque `from()` liste les champs un par un.

adapter/in/web/security — `SessionCookieAuthenticationFilter` lit le cookie, vérifie le jeton signé, et pose un `AuthenticatedUser` dans le contexte. **Aucun accès base par requête** : l'identité et le rôle voyagent dans le jeton.

adapter/in/web/WebExceptionHandler — le seul endroit qui associe une catégorie d'exception à un code HTTP. Toutes les erreurs sortent au même format : `{ timestamp, status, error, message, path, details }`.

adapter/in/event — `LabProvisioningListener` provisionne le lab à l'inscription, **dans la même transaction** que la création du compte.

adapter/in/startup — trois `ApplicationRunner` : semis du catalogue de machines, semis des cours, promotion des comptes administrateurs.

Sortants

Paquet	Contenu
<code>out/persistence</code>	Entités JPA, dépôts Spring Data, adaptateurs de traduction
<code>out/security</code>	<code>BCryptPasswordHasher</code> , <code>JwtTokenService</code> , <code>SecureRandomSecretGenerator</code>
<code>out/hypervisor</code>	<code>SimulatedHypervisorAdapter</code> (défaut) et <code>docker/</code> (conteneurs réels)
<code>out/vpn</code>	<code>EasyRsaCertificateAuthority</code> — autorité de certification intégrée
<code>out/transaction</code>	<code>SpringTransactionAdapter</code>
<code>out/event</code>	<code>SpringDomainEventPublisher</code>
<code>out/notification</code>	<code>LoggingPasswordResetNotifier</code> (à remplacer par un vrai envoi)
<code>out/process</code>	Exécution de commandes externes, avec délai maximal

Les entités JPA ne sortent jamais de leur adaptateur. Chaque adaptateur de persistance traduit entité ↔ agrégat dans deux méthodes privées `toEntity` / `toDomain`. C'est un peu de code répétitif, et c'est ce qui garde le domaine libre de JPA.

Le choix de l'hyperviseur se fait par `@ConditionalOnProperty` sur `app.hypervisor.mode` : `simulated` par défaut, `docker` pour des conteneurs réels. Brancher Proxmox ou vSphere revient à écrire une troisième classe.

7. La racine de composition

`config/UseCaseConfig` est le seul fichier où les cas d'usage rencontrent leurs adaptateurs. Les services y sont construits à la main :

```
@Bean
public BoxService boxService(BoxRepositoryPort boxes, OwnRepositoryPort owns,
                             BoxRatingRepositoryPort ratings,
                             GetPlayerProgressUseCase progress,
                             TransactionPort transactions, Clock clock) {
    return new BoxService(boxes, owns, ratings, progress, transactions, clock);
}
```

Pourquoi ne pas annoter `BoxService` avec `@Service` ? Parce que le jour où l'on veut l'instancier dans un test, un batch ou un autre point d'entrée, il suffit d'appeler son constructeur. Le prix est une méthode `@Bean` par service ; le gain est un cœur métier réellement indépendant.

`Clock` est injecté partout plutôt que `Instant.now()` : les tests figent le temps, et aucune règle ne dépend de l'horloge de la machine.

8. Le cycle de vie d'une requête

Suivons `POST /api/boxes/sentinel/flags` — un joueur soumet un flag. C'est le meilleur trajet pour comprendre l'ensemble.

#	Où	Ce qui se passe
1	Nginx	Sert l'application, relaie <code>/api</code> vers le backend (même origine).
2	<code>SessionCookieAuthenticationFilter</code>	Lit le cookie <code>LAB_SESSION</code> , vérifie la signature, pose <code>AuthenticatedUser(id, email, role)</code> .
3	<code>SecurityConfig</code>	La route n'est ni publique ni <code>/api/admin/**</code> : une session suffit.
4	<code>BoxController.submit</code>	Valide la forme du corps (<code>@Valid</code>), convertit le principal en <code>Actor</code> , appelle <code>SubmitFlagUseCase</code> .
5	<code>BoxService.submitFlag</code>	Vérifie le format du flag avant toute requête ; charge la machine (404 sinon).
6	<code>TransactionPort</code>	Ouvre une transaction courte.
7	<code>OwnRepositoryPort</code>	Ce flag est-il déjà validé par ce joueur (409) ? Personne ne l'a-t-il validé avant (first blood) ?
8	<code>Box.claim(...)</code>	Le domaine décide : compare le flag en temps constant, refuse (400) ou frappe un <code>Own</code> avec les points de la difficulté.
9	<code>OwnRepositoryPort.save</code>	Enregistre. L'unicité (joueur, machine, flag) est tenue en base : deux soumissions simultanées ne comptent qu'une fois.
10	<code>ScoreboardService</code>	Recalcule la progression (points, rang, machines possédées).
11	<code>FlagSubmissionResponse.from</code>	Mappage explicite : ni flag, ni empreinte dans la réponse.
12	<code>WebExceptionHandler</code>	En cas d'exception métier, traduit la catégorie en statut et en corps normalisé.

Chaque couche ne fait que son métier : le contrôleur ne connaît pas les points, le service ne connaît pas les flags, le domaine ne connaît pas HTTP.

9. La base de données

Neuf tables, décrites dans `backend/src/main/resources/db/schema.sql`.

Table	Rôle
<code>app_user</code>	Comptes, rôle, jeton de réinitialisation (empreinte seule)
<code>virtual_machine</code>	Machines personnelles et leurs accès temporaires
<code>vpn_profile</code>	Certificat client courant de chaque utilisateur
<code>box</code>	Catalogue des machines à compromettre (empreintes des flags)
<code>box_own</code>	Flags validés, points figés, first blood
<code>box_rating</code>	Difficulté ressentie, un vote par joueur et par machine
<code>box_instance</code>	Cibles lancées à la demande, avec leur échéance
<code>writeup</code>	Comptes rendus, un par auteur et par machine
<code>quiz_question</code> / <code>quiz_choice</code>	Questions de quiz et leurs propositions
<code>media_asset</code>	Fiches des vidéos téléversées (le contenu vit sur le disque)
<code>course</code> / <code>course_section</code>	Cours et leurs sections (texte, vidéo)
<code>course_section_completion</code>	Suivi de lecture

Le schéma n'est pas généré par Hibernate (`ddl-auto: none`). Il est écrit à la main, en SQL compatible PostgreSQL et H2, et **idempotent** : PostgreSQL l'exécute à la création du volume, le backend le rejoue sans effet à chaque démarrage. Une évolution s'ajoute en fin de fichier sous une forme rejouable (`ALTER TABLE ... ADD COLUMN IF NOT EXISTS ...`).

Les invariants du domaine sont doublés en base quand c'est possible : `ck_vm_state` interdit une machine arrêtée avec des accès, et `uk_own_user_box_kind` interdit de compter deux fois les mêmes points. Le domaine protège du bogue, la base protège de la concurrence.

10. Sécurité

Sujet	Décision
Session	JWT (8 h) dans un cookie <code>HttpOnly</code> , <code>SameSite=Strict</code> , limité à <code>/api</code> . Illisible en JavaScript, jamais renvoyé dans un corps de réponse.
CSRF	Protection par jeton désactivée : <code>SameSite=Strict</code> empêche l'envoi depuis un autre site, et Nginx sert l'application et l'API sur la même origine. Si la politique SameSite est assouplie, réactiver CSRF.
Mots de passe	BCrypt coût 12. La connexion prend le même temps que le compte existe ou non (empreinte factice), ce qui empêche d'énumérer les comptes au chronomètre.
Réinitialisation	Jeton stocké en SHA-256 seulement, expiration 15 minutes, usage unique.
Flags	Empreinte SHA-256, comparaison en temps constant, jamais dans une réponse ni dans un journal.
Autorisation	<code>/api/admin/**</code> exige le rôle ADMIN dans la chaîne de sécurité et chaque cas d'usage le revérifie (<code>AdminPolicy</code>). Une règle métier ne dépend pas de la configuration d'un framework.
Rôle administrateur	Vient de <code>APP_ADMIN_EMAILS</code> et de là seulement. Aucune route ne l'accorde, pas même à un administrateur.
Discrétion	Une ressource d'autrui répond 404, pas 403 : le refus ne révèle pas l'existence.
Vidéos	Seules <code>http(s)</code> sont acceptées ; seules YouTube et Vimeo sont intégrées dans la page, la même liste que la CSP servie par Nginx.
En-têtes	Nginx ajoute CSP, <code>X-Frame-Options</code> , <code>nosniff</code> .

11. Configuration

Tout passe par `AppProperties` (préfixe `app`), typée et documentée.

```
app
├─ jwt          secret, validity, issuer
├─ session      cookieName, cookieSecure, cookieSameSite
├─ cors         allowedOrigins
├─ security     exposeResetTokenInResponse, resetTokenValidity, adminEmails
├─ boxes        logSeededFlags, leaderboardSize
├─ hypervisor   mode, defaultUsername, simulated*, docker{...}
└─ vpn          enabled, udpHost, tcpHost, labNetwork, easysrsa*, ...
```

Trois profils : **défaut** (PostgreSQL), **local** (H2 en mémoire, flags des cours journalisés), **test** (H2, compte admin de test).

Les variables d'environnement importantes sont documentées dans le `README` et dans `.env.example`. Deux réglages ne doivent jamais rester à leur valeur de démonstration en production : `APP_JWT_SECRET` et `APP_EXPOSE_RESET_TOKEN`.

12. Tests

126 tests backend, 42 frontend. La répartition est volontaire.

Niveau	Où	Ce qu'il protège	Coût
Domaine	domain/**Test	Les règles : machine à états, barème, rangs, format des flags	quelques ms
Service	application/service/**Test	L'orchestration, avec des doublures en mémoire (InMemory*)	quelques ms
Intégration	adapter/in/web/**IntegrationTest	Le trajet complet : HTTP, sécurité, JPA, schéma SQL réel	quelques secondes

Les doublures vivent dans `application/fakes/` et se comportent comme la vraie base (copies défensives, séquences d'identifiants, contraintes d'unicité). Un service se teste donc sans Spring, sans base, et sans lenteur.

La suite tourne deux fois en intégration continue : sur H2, puis sur PostgreSQL. Ce n'est pas un luxe. Un `@Lob` sur une chaîne écrit un *large object* sur PostgreSQL et ne range que son identifiant dans la colonne, là où H2 range le texte sans broncher : le bogue n'existait que sur la base de production. Pour rejouer la suite sur PostgreSQL en local :

```
TEST_DB_URL=jdbc:postgresql://localhost:5432/labplatform_test \
TEST_DB_USERNAME=labplatform TEST_DB_PASSWORD=labplatform mvn test
```

13. Le frontend en deux mots

React 19 + TypeScript, en Clean Architecture — le même principe, les mêmes frontières.

```
src/
├─ domain/          Aucune dépendance à React ni à HTTP
│  └─ models/       User, VirtualMachine, Box, Progress, Course, Profile, Admin, Video
│  └─ repositories/ Interfaces (AuthRepository, BoxRepository, CourseRepository...)
│  └─ usecases/      Login, StartVm, SubmitFlag, ToggleSection, SaveCourse...
│  └─ validation/    Règles de saisie, miroir des règles serveur
├─ data/            Implémentations HTTP des repositories (axios) + mappers DTO
├─ di/container.ts   Racine de composition : seul fichier qui connaît les implémentations
├─ presentation/     design-system, layouts, navigation, features, hooks, pages, styles
└─ app/App.tsx       Routage
```

La correspondance avec le backend est directe : `domain/` ↔ `domain/`, `usecases/` ↔ `port/in`, `repositories/` ↔ `port/out`, `data/` ↔ `adapter/out`, `container.ts` ↔ `UseCaseConfig`.

Deux points à connaître : **la validation est doublée** côté client pour répondre sans aller-retour, mais **le serveur reste l'autorité** ; et le menu filtre les entrées par rôle, ce qui est un **confort d'affichage**, jamais une protection.

14. Recettes d'évolution

Ajouter un cas d'usage (le cas courant)

1. **Domaine** — si une règle nouvelle apparaît, elle va dans un agrégat ou un objet-valeur, avec son test.
2. **port/in** — une interface, une méthode, prenant `Actor` en premier paramètre.
3. **port/out** — seulement si un besoin technique nouveau apparaît.
4. **service** — l'implémentation, sans annotation Spring.
5. **UseCaseConfig** — une méthode `@Bean`.
6. **Contrôleur + DTO** — mappage explicite, champs listés un par un.
7. **Tests** — un test de service avec les doublures, plus un cas dans le test d'intégration si la route est nouvelle.
8. **Frontend** — modèle, interface de dépôt, cas d'usage, implémentation HTTP, câblage dans `container.ts`, hook, page.

Brancher un hyperviseur réel (Proxmox, vSphere, cloud)

Écrire une classe qui implémente `HypervisorPort`, l'annoter `@ConditionalOnProperty(prefix = "app.hypervisor", name = "mode", havingValue = "proxmox")`, ajouter ses réglages dans `AppProperties.Hypervisor`. **Aucun cas d'usage ne change.** C'est exactement ce que fait déjà l'adaptateur Docker.

Brancher un envoi d'e-mails

Implémenter `PasswordResetNotifieurPort` à la place de `LoggingPasswordResetNotifieur`, puis passer `APP_EXPOSE_RESET_TOKEN=false`.

Ajouter une filière de cours

Une constante dans `domain/academy/Track`, une entrée dans `PRIMARY_NAV` et dans `TRACK_PAGES` côté frontend. Le reste suit : API, pages, avancement.

Ajouter un haut fait

Une constante dans `domain/achievement/Achievement`, avec sa condition. Rien d'autre : ni table, ni migration, ni recalcul.

Ajouter un champ à une entité

1. La colonne dans `schema.sql`, et un `ALTER TABLE ... IF NOT EXISTS` en fin de fichier pour les bases existantes.
2. Le champ dans l'entité JPA — **pas de `@Lob` sur une chaîne (§12)**.
3. Le champ dans l'objet du domaine, avec sa validation.
4. Les deux traductions dans l'adaptateur de persistance.
5. Le champ dans le DTO, si l'extérieur doit le voir.
6. Un test d'intégration qui le lit et l'écrit, joué sur PostgreSQL.

Ajouter une entrée de menu

Une ligne dans `frontend/src/presentation/navigation/navigation.ts`. Le menu accepte des groupes à deux niveaux et un filtre par rôle (`roles: [ 'ADMIN' ]`). La `Sidebar` n'est pas à modifier.

15. Pièges connus

Piège	Pourquoi	Quoi faire
<code>@Lob</code> sur une <code>String</code>	PostgreSQL écrit une <i>large object</i> et ne stocke que son identifiant ; illisible hors transaction. H2 ne le reproduit pas.	<code>columnDefinition = "text"</code> , et rejouer la suite sur PostgreSQL.
Tester seulement sur H2	Les différences de dialecte ne se voient qu'en production.	La CI rejoue tout sur PostgreSQL ; faites-le aussi en local avant une revue.
Appeler l'hyperviseur dans une transaction	Un démarrage prend des secondes et bloque une connexion SQL.	Agir hors transaction, puis rouvrir une transaction courte.
Sérialiser un agrégat	<code>Box</code> porte ses flags, <code>User</code> son empreinte, <code>QuizChoice</code> la bonne réponse.	Mappage DTO explicite, toujours.
Répondre 403 au lieu de 404	Un 403 révèle l'existence de la ressource d'autrui.	404 pour tout ce qui n'appartient pas à l'appelant.
Recalculer des points acquis	Rééquilibrer une machine réécrirait le passé.	Les points sont figés dans <code>Own</code> à la validation.
Annoter un service <code>@Service</code>	Le cœur redevient dépendant du framework.	Une méthode <code>@Bean</code> dans <code>UseCaseConfig</code> .

16. Par où commencer

Pour une première heure dans le code, dans cet ordre :

1. `domain/lab/VirtualMachine.java` — un agrégat court, avec un invariant explicite. Le style de tout le domaine y est visible.
2. `application/service/LabService.java` — l'orchestration, et la discipline transactionnelle.
3. `config/UseCaseConfig.java` — comment tout est câblé.
4. `adapter/in/web/BoxController.java` + `dto/BoxDtos.java` — la frontière HTTP et le mappage explicite.
5. `application/service/BoxServiceTest.java` — comment on teste un cas d'usage sans rien démarrer.

Puis lancer la plateforme et la parcourir :

```
cd backend && mvn spring-boot:run -Dspring-boot.run.profiles=local
cd frontend && npm install && npm run dev
```

17. Glossaire

Terme	Sens ici
Agrégat	Objet du domaine responsable d'un invariant, seule porte d'entrée pour le modifier (<code>Box</code> , <code>Course</code> , <code>User</code> , <code>VirtualMachine</code>).
Objet-valeur	Objet sans identité, valide dès sa construction, immuable (<code>Email</code> , <code>Flag</code> , <code>ConnectionInfo</code>).
Port	Interface appartenant à l'application : entrant = ce qu'elle sait faire, sortant = ce dont elle a besoin.
Adaptateur	Réalisation technique d'un port, à la périphérie.
Racine de composition	L'unique endroit qui connaît les implémentations concrètes (<code>UseCaseConfig</code> , <code>container.ts</code>).
Actor	L'appelant d'un cas d'usage : identifiant et rôle.
Box	Machine du catalogue, à compromettre. À ne pas confondre avec <code>VirtualMachine</code> , la machine d'attaque personnelle.
Flag	Preuve de compromission, 32 caractères hexadécimaux, stockée en empreinte.
First blood	Premier joueur à valider un flag donné.
Filière (Track)	Regroupement de cours : forensique ou défense.